Project Architecture and Conceptual Overview

Autonomous Reinforcement Learning for Adaptive Resonator Calibration: A Classical Analogue for Quantum Control Systems

Project Architecture and Conceptual Overview

1. Project Overview

- The objective of this project is to build a closed-loop autonomous control system capable of learning how to optimally excite a drifting resonator despite environmental noise and parameter instability.
- The system simulates the control architecture used in modern quantum hardware calibration systems, where microwave control pulses must be continuously tuned to maintain high-fidelity quantum gate operations.
- This project implements a simplified but conceptually accurate version of such a system using classical electronics.

---

2. System Architecture

The system consists of three tightly coupled layers that form a continuous feedback loop.

- **Physical Layer**
  - Represents the hardware system being controlled.
  - Components include:
    - LC resonator
    - Varactor diode for frequency tuning

        - Noise injection circuit
        - Signal input and output measurement path
    - This layer behaves similarly to a quantum resonator or qubit device whose properties drift over time.

- **Control Layer**
    - Handles real-time hardware interaction.
    - Responsibilities include:
        - generating control pulses
        - measuring resonator response
        - communicating with the optimization software
    - This layer is implemented using a microcontroller such as an ESP32.

- **Intelligence Layer**
    - This layer performs autonomous optimization.
    - Responsibilities include:
        - analyzing measurement data
        - adjusting pulse parameters
        - learning optimal control strategies
    - The intelligence layer is implemented in Python using machine learning algorithms.

---

3. System Feedback Loop

The complete system operates as a closed feedback loop. The AI continuously modifies pulse parameters in order to maximize the resonator's response. The flow operates as follows:

- AI Optimizer generates pulse parameters.
- The embedded controller uses these to output a generated waveform.

Resonator Hardware produces a measured response.
- The embedded controller translates this response into a reward signal.
- AI Optimizer receives the reward and adjusts parameters accordingly.

---

4. Core Objective of the Project

The system must demonstrate adaptive optimal control in a non-stationary physical environment. Specifically, the system should:

- detect changes in resonator behavior
- adapt control pulses automatically
- maintain optimal excitation conditions despite drift

The system does not have direct access to internal physical variables. It must infer the optimal control strategy using only input–output observations.

---

5. Physical Principle: Resonance

- The resonator is constructed using an LC circuit.
- An LC circuit stores energy by oscillating between magnetic energy in the inductor and electric energy in the capacitor.
- The resonant frequency of the circuit is given by:

  $$f_0 = \frac{1}{2\pi\sqrt{LC}}$$

- Where L is the inductance and C is the capacitance.
- At this frequency, the circuit exhibits maximum energy transfer.
- When driven at frequencies far from resonance, the response amplitude decreases significantly.

---

6. Simulating Environmental Drift

- Real quantum systems experience frequency drift caused by environmental noise.
- Common sources include magnetic field fluctuations, temperature changes, electrical noise, and device imperfections.
- To replicate this behavior, the resonator's capacitance is made variable.
- This is achieved using a varactor diode.

---

7. Varactor-Based Frequency Control

- A varactor diode is a semiconductor device whose capacitance depends on the applied reverse bias voltage.
- Relationship: A higher reverse voltage yields lower capacitance, while a lower reverse voltage yields higher capacitance.

- Since the resonant frequency depends on capacitance, changing the bias voltage changes the resonant frequency:

  $$f_0 = \frac{1}{2\pi\sqrt{L C(V)}}$$

- By injecting a slow noise signal into the bias voltage, the resonant frequency will drift over time.

---

8. Control Pulses

- The resonator is excited using control pulses.
- A control pulse is a time-dependent voltage waveform applied to the input of the resonator.
- Each pulse is defined by several parameters:
  - carrier frequency
  - amplitude
  - pulse duration
  - waveform envelope
  - phase
- These parameters determine how effectively the pulse transfers energy into the resonator.

---

9. Pulse Envelope Shapes

The envelope defines the time-domain shape of the pulse. Common envelope shapes include:

- Square Pulse: Simple on-off waveform.
- Gaussian Pulse: Smooth bell-shaped waveform that reduces spectral leakage.
- Cosine Pulse: Smooth waveform with sinusoidal envelope.
- DRAG Pulse: Derivative-based waveform used in quantum gate control to reduce leakage errors.

Although the resonator in this project is classical, implementing these pulse shapes demonstrates advanced control techniques used in quantum computing.

---

10. Measurement of Resonator Response

After a pulse is applied, the system measures the resonator's response. Possible measurement metrics include:

- output signal amplitude
- phase shift
- stored energy
- ring-down decay rate

For simplicity, this project focuses primarily on measuring output amplitude. Higher amplitude indicates that the pulse frequency closely matches the resonant frequency.

---

11. Reward Function

The reinforcement learning system requires a numerical reward signal. The reward represents how effective the chosen pulse parameters were.

- A simple reward function is: Reward = measured response amplitude.
- A more advanced formulation may include power penalties: Reward = energy transferred – penalty(pulse power).
- The reward function must satisfy three conditions: consistent measurement, monotonic relation with system performance, and tolerance to noise.

---

12. Reinforcement Learning Framework

The control problem is formulated as a reinforcement learning task. The RL framework consists of the following components:

- **State:** The information available to the agent before selecting an action. Possible state variables include recent amplitude measurements, previously used pulse parameters, and the estimated resonant frequency.
- **Action:** The parameters selected by the agent. Example action variables include pulse frequency, pulse amplitude, pulse duration, and waveform type.
- **Reward:** A numerical score indicating pulse performance. The agent attempts to maximize the reward.
- **Episode:** A sequence of optimization steps during which the agent interacts with the resonator. Episodes may reset when system parameters drift too far, optimization stagnates, or the experiment is restarted.

---

13. Why Reinforcement Learning Is Used

- Traditional calibration methods rely on brute-force parameter sweeps, testing many frequencies to identify the maximum response.
- However, when multiple parameters must be optimized simultaneously, exhaustive search becomes inefficient.
- Reinforcement learning offers several advantages: adaptive parameter exploration, robustness to measurement noise, and the ability to track continuously changing environments.

---

14. System Identification

Before reinforcement learning can operate effectively, the resonator must be characterized. This process is known as system identification.

- The controller performs a frequency sweep.
- Example sweep range: 9.8 MHz to 10.2 MHz.
- For each test frequency, the system records the response amplitude.
- The result is the resonator's frequency response curve.

---

15. Quality Factor (Q Factor)

The quality factor describes how sharply the resonator responds to frequency.

$$Q = \frac{f_0}{\Delta f}$$

Where f_0 is the resonance frequency and Δf is the bandwidth of the resonance peak. A high Q factor indicates a sharp resonance peak, strong frequency selectivity, and sensitivity to small frequency shifts.

---

16. Pulse Shaping

Simple square pulses generate broad spectral components. Quantum control systems often use smooth pulse envelopes to reduce unwanted frequencies.

- Example Gaussian pulse:

  $$A(t) = A_0 e^{-\frac{(t-t_0)^2}{2\sigma^2}}$$

- Smooth pulses produce cleaner frequency responses and reduce distortion.

---

17. Closed-Loop Optimization Process

The system continuously repeats the following steps indefinitely:

- The AI selects pulse parameters.
- The controller generates the waveform.
- The resonator responds to the pulse.
- The measurement system records the response.
- The reward is calculated.
- The AI updates its control policy.

As the resonator drifts, the AI continuously adapts.

---

18. Control Loop Timing

The optimization loop must operate faster than the resonator drift rate.

- Example timing breakdown: Pulse generation (~10 μs), Measurement (~50 μs), Communication (~1 ms), and AI update (~1–10 ms).
- Typical optimization step duration: 10–50 ms.

---

19. Simulation Environment

Before hardware testing begins, the system should be simulated.

- The resonator can be modeled using a Lorentzian response function:

  $$A(f) = \frac{1}{1 + Q^2\left(\frac{f-f_0}{f_0}\right)^2}$$

- Frequency drift can be simulated using random perturbations:

  $$f_0(t) = f_0 + noise(t)$$

- Simulation allows the development of the optimization algorithm without hardware constraints.

---

20. Expected Experimental Outputs

The completed system should produce measurable results, including:

- Resonator Frequency Drift: A time-series plot showing changes in resonance frequency.
- AI Parameter Adaptation: Graphs showing pulse parameters adjusting over time.
- Energy Transfer Stability: Measured amplitude remaining near optimal values despite drift.
- Optimization Convergence: Reward improving as the RL agent learns.

---

21. Scientific Significance

This project demonstrates several key concepts used in modern quantum hardware control systems. These include:

- adaptive calibration
- closed-loop feedback control
- noise-resilient optimization
- integration of machine learning with experimental hardware

Although the hardware system is classical, the architecture closely resembles control stacks used in quantum computing laboratories.

Complete Hardware and Software Requirements

Complete Hardware and Software Requirements

1. Overview

- This section lists all required components and tools for building the autonomous adaptive resonator control system.
- The system requires components across five main categories:
  - Control hardware
  - Analog signal chain
  - Resonator hardware
  - Measurement hardware
  - Software stack
- Each category must be carefully designed to ensure stable measurements, low noise, and reliable control loop operation.

---

2. Control Hardware

- The control hardware is responsible for:
  - generating pulse waveforms
  - sampling measurement signals
  - communicating with the AI optimizer
- The controller also handles timing-sensitive operations.

Primary Microcontroller

- Recommended controller: ESP32 development board
- Typical specifications:
  - Dual-core processor (up to 240 MHz)
  - WiFi / Bluetooth connectivity
  - SPI / I2C / UART interfaces
  - hardware timers

    - sufficient RAM for signal buffering
- The ESP32 acts as the bridge between hardware and AI software.

---

3. External Data Converters

- External converters improve signal accuracy and stability.
- The internal ESP32 converters are not ideal for precision measurements.

External DAC (Waveform Generator)

- The DAC generates pulse waveforms.
- Recommended components: MCP4921
- Specifications:
    - 12-bit resolution
    - SPI interface
    - sampling rate up to ~350 kSPS
    - single-channel output
- Alternative options:
    - MCP4922 (dual channel)
    - AD5662 (higher precision)

External ADC (Signal Measurement)

- The ADC measures resonator output amplitude.
- Recommended components: MCP3202
- Specifications:
    - 12-bit resolution
    - SPI interface
    - sampling rate up to ~100 kSPS
    - two input channels
- Alternative options:
    - MCP3204 (4 channels)

    - ADS1115 (16-bit but slower)
- SPI-based ADCs are preferred due to higher throughput.

---

4. Analog Signal Conditioning

- The analog signal chain ensures the generated pulses are clean and that measurements are stable.

Operational Amplifiers

- Op-amps are required for:
    - waveform buffering
    - signal amplification
    - impedance matching
- Recommended general-purpose op-amps: LM358, TL072, OPA2134 (higher performance)

Low-Pass Filters

- Low-pass filters remove high-frequency noise before ADC measurement.
- Typical configuration: RC filter
- Example values: R = 10 kΩ, C = 10 nF
- These values may be adjusted depending on system bandwidth.

---

5. Resonator Hardware

- The resonator forms the physical system being controlled.

Inductor

- The inductor determines the base resonance frequency.

- Typical values: 1 μH – 10 μH
- Inductors with a high Q-factor are preferred.
- Example types: shielded RF inductors, air-core inductors

Varactor Diode

- The varactor enables voltage-controlled capacitance.
- Recommended components: MV209, BB909, SMV1231
- These diodes allow resonance tuning through bias voltage.

Biasing Network

- The varactor requires a bias voltage circuit.
- Typical components:
    - bias resistor (10 kΩ – 100 kΩ)
    - bypass capacitor
    - voltage divider or DAC control
- The bias voltage determines the instantaneous capacitance.

Noise Injection Circuit

- To simulate environmental drift, a slow noise signal is injected into the varactor bias.
- Possible implementations:
    - filtered PWM signal from the microcontroller
    - low-frequency oscillator
    - noise diode circuit
- This introduces controlled frequency drift.

---

6. Pulse Generation Circuit

- The pulse generator converts the DAC output into a usable excitation signal.
- Components required:
  - DAC output stage
  - op-amp buffer
  - optional amplification stage
- The buffer ensures the DAC signal can drive the resonator without distortion.

---

7. Measurement Circuit

- The measurement path captures resonator response.
- Components include:
  - signal tap from resonator output
  - amplifier stage
  - low-pass filter
  - ADC input
- The goal is to measure a stable representation of the resonator amplitude.

---

8. Power Supply System

- Stable power is essential for analog measurements.
- Recommended components:
  - Voltage regulators: AMS1117-3.3, LM317 adjustable regulator
  - Filtering components: 10 μF electrolytic capacitors, 100 nF ceramic capacitors
- Separate analog and digital ground paths should be used when possible.

---

9. Additional Hardware

- Additional components required for assembly and debugging.
- These include: Breadboard or PCB, Jumper wires, Coaxial cables for signal integrity, Oscilloscope (optional but highly recommended), Multimeter, USB power supply

---

10. Software Requirements

- The software stack consists of several layers.

Embedded Firmware

- Firmware runs on the ESP32 and controls hardware operations.
- Responsibilities include:
    - DAC waveform generation
    - ADC measurement
    - communication with Python
    - timing control
- Development environments: Arduino IDE, PlatformIO, ESP-IDF

Python Control Software

- The AI optimization system runs in Python.
- Required responsibilities:
    - sending pulse parameters
    - receiving measurement data
    - computing rewards
    - running reinforcement learning algorithms

Python Libraries

- Essential Python libraries include:

- Numerical computation: numpy, scipy
- Machine learning: PyTorch, TensorFlow (optional), stable-baselines3 (RL library)
- Visualization: matplotlib, seaborn
- Communication: pyserial

---

11. Data Logging System

- All experiment data should be recorded for analysis.
- Data fields may include: timestamp, pulse frequency, pulse amplitude, pulse duration, waveform type, measured amplitude, reward value
- Data can be stored in: CSV files, SQLite database, pandas dataframes

---

12. Simulation Environment

- Before hardware testing begins, the system should be simulated.
- Simulation components include: resonator transfer function model, frequency drift generator, measurement noise model, reinforcement learning environment
- Simulation tools: Python, NumPy, SciPy

---

13. Visualization Dashboard

- A dashboard should display real-time system behavior.
- Recommended displays include: Resonator response plot, Pulse parameter evolution, Reward vs time, Frequency sweep results, Drift tracking visualization
- Framework options: matplotlib, Plotly, Dash, PyQt

14. Development Tools

- Additional tools improve the development workflow.
- Recommended tools:
    - Version control: Git
    - Code editor: VS Code
    - Hardware debugging: Serial monitor, logic analyzer (optional)

---

15. Summary of System Components

- The full system consists of: Control hardware, External ADC and DAC modules, Analog signal conditioning circuits, LC resonator with varactor tuning, Noise injection system, Measurement electronics, Embedded firmware, Python AI optimizer, Visualization and logging software
- These components collectively form the adaptive closed-loop control system.

Project Execution Roadmap

Project Execution Roadmap

Phase 1 — System Specification and Parameter Selection

- **Objective**: Define the operating parameters of the system before building any hardware or software. Most project failures occur because the operating range is chosen incorrectly.

- **Key Parameters to Define:**

  - Resonator Frequency Range: Choose a frequency that is easy to generate with ESP32 hardware, measurable using available ADC hardware, and compatible with affordable components. Recommended range: 200 kHz – 1 MHz. Avoid frequencies above ~2 MHz during the first implementation.
  - Resonator Q-Factor: Target Q factor: 20 – 100. Very high Q resonators (>1000) are difficult to stabilize.
  - Frequency Drift Range: Define the drift magnitude. Example: ±5% frequency variation. Example: Nominal resonance = 500 kHz, Drift range = 475 kHz – 525 kHz.
  - Pulse Duration: Recommended pulse duration: 5 μs – 50 μs. This allows several oscillation cycles.
  - Measurement Sampling Rate: Minimum recommended sampling rate: 50 kHz – 200 kHz.

- **Deliverables**: Document containing resonance frequency target, drift range, pulse duration, and measurement sampling rate.
- **Validation Criteria:** Parameters must be consistent with ESP32 timing capabilities, DAC update speed, and ADC sampling speed.

Phase 2 — Resonator Simulation

- **Objective:** Create a software model of the resonator. The reinforcement learning system must be tested in simulation before hardware is used.
- **Resonator Model:** The resonator response can be approximated by a Lorentzian function.

  $$A(f) = \frac{1}{1 + Q^2 \left(\frac{f-f_0}{f_0}\right)^2}$$
  Where f = input frequency, f_0 = resonant frequency, and Q = quality factor.

- **Drift Model:** Introduce time-varying drift. Example model:

  $$f_0(t) = f_0 + noise(t)$$
  Noise sources can include random walk, sinusoidal drift, and filtered noise.

- **Measurement Noise:** Add simulated measurement noise: measured_response = true_response + gaussian_noise. Noise level example: σ = 2–5% signal amplitude.
- **Deliverables:** Python module implementing resonator model, drift generator, and noisy measurement output.
- **Validation Criteria:** Simulation must produce realistic resonance curves, observable drift, and noisy measurements.

Phase 3 — Reinforcement Learning Environment

- **Objective:** Build the RL environment that interacts with the resonator simulation.
- **Environment Structure:** The environment must expose the following functions: reset(), step(action), and observe_state().

- **State Definition:** State variables may include previous pulse frequency, previous pulse amplitude, recent response measurements, and estimated resonance frequency.
- **Action Space:** Actions represent pulse parameter changes. Example variables: pulse frequency, pulse amplitude, pulse duration, and pulse waveform type.
- **Reward Function:** Initial reward definition: reward = measured amplitude. Optional improvements: reward = amplitude – penalty(power).
- **RL Algorithms: Recommended algorithms:** PPO (Proximal Policy Optimization) and Bayesian Optimization (simpler alternative). For early development, Bayesian optimization may be easier.
- **Deliverables:** Python RL environment capable of interacting with the simulated resonator.
- **Validation Criteria:** The RL agent must learn to identify resonance frequency, track simulated drift, and maintain high response amplitude.

Phase 4 — Hardware Resonator Construction

- **Objective:** Build the physical LC resonator.
- **Circuit Design:** Basic resonator configuration: Input signal → LC network → output measurement. Capacitance is controlled using a varactor diode.
- **Component Selection:** Example starting values: Inductor = 10 μH, Varactor capacitance = 20 pF – 200 pF. Approximate resonance calculation:

  $$f_0 = \frac{1}{2\pi\sqrt{LC}}$$

Example result: ~500 kHz resonance.

- **Drift Injection:** Apply a slowly varying voltage to the varactor bias. Possible methods: filtered PWM, noise generator, DAC output. This creates frequency drift.

- **Deliverables:** Working resonator circuit.
- **Validation Criteria:** Frequency sweep must produce a clear resonance peak and measurable drift when bias voltage changes.

Phase 5 — Pulse Generator Development

- **Objective:** Build the waveform generation system.

- **Waveform Generation Method:** Waveforms are produced using: ESP32 → SPI DAC → analog buffer → resonator.

- **Pulse Types to Implement:** Minimum required waveforms: square pulse, Gaussian pulse, and cosine pulse. Waveforms are represented as sample arrays. Example Gaussian pulse: [0, 4, 15, 40, 80, 120, 150, 120, 80, 40, 15, 4, 0]. These values are streamed to the DAC.

- **Timing Control:** Use ESP32 hardware timers, not software delays.

- **Deliverables:** Firmware capable of generating programmable pulses and adjusting frequency and amplitude.

- **Validation Criteria:** Waveforms must appear correctly on an oscilloscope.

Phase 6 — Measurement System

- **Objective:** Measure resonator response reliably.
- **Measurement Pipeline:** resonator output → amplifier → low-pass filter → ADC → ESP32.
- **Signal Processing:** Perform oversampling, averaging, and basic filtering. Example: average 64 ADC samples.
- **Deliverables:** Firmware capable of returning stable amplitude measurements.
- **Validation Criteria:** Measurement noise should be <5%.

Phase 7 — Hardware Control Loop

- **Objective:** Connect Python control software to hardware.
- **Communication Protocol:** Use serial communication. Example speed: 921600 baud.
- **Data Exchange Format:** Example command: `SET frequency amplitude duration waveform`. Example response: `MEASURE amplitude`.
- **Deliverables:** Python script capable of controlling ESP32 and collecting measurements.
- **Validation Criteria:** Control loop latency <50 ms.

Phase 8 — AI Integration with Hardware

- **Objective:** Replace the simulated environment with real hardware.
- **Integration Steps:** RL agent selects pulse parameters, parameters sent to ESP32, pulse executed, measurement returned, reward calculated, and RL agent updates policy.
- **Deliverables:** AI agent controlling physical resonator.
- **Validation Criteria:** The system must track resonator drift.

Phase 9 — Visualization Dashboard

- **Objective:** Create real-time monitoring tools.
- **Required Visualizations:** Resonator response vs frequency, Pulse parameter evolution, Reward vs time, Estimated resonance frequency, and Drift tracking.
- **Implementation Options:** Possible frameworks: matplotlib, Plotly, Dash, and PyQt.

Phase 10 — Experimental Validation

- **Objective:** Demonstrate system performance.
- **Required Experiments:**
    - **Drift Tracking Test:** Introduce controlled drift and measure AI's ability to maintain resonance.
    - **Pulse Optimization Test:** Start with random pulse parameters and observe convergence toward optimal values.

- **Noise Robustness Test:** Increase measurement noise and observe system stability.

- **Deliverables:** Final report including response graphs, convergence plots, and drift compensation performance.

- **Final Outcome:** If the system is functioning correctly, the AI should be able to continuously identify the resonator's resonant frequency, adjust pulse parameters in real time, and maintain high energy transfer despite frequency drift. This demonstrates autonomous adaptive control of a physical system.

Hardware and Software Architecture

Hardware Circuit Architecture and Detailed Schematic

1. Hardware Architecture Overview

- The physical hardware consists of four major subsystems:
  - Pulse Generation System
  - Resonator Circuit
  - Measurement and Signal Conditioning
  - Controller Interface
- Each subsystem must be designed carefully to maintain signal integrity and measurement reliability.

System Block Diagram

- Python AI --> Serial Communication --> ESP32 Controller
- ESP32 Controller --> SPI Interface --> DAC (MCP4921)
- DAC --> Analog Buffer --> Pulse Injection --> LC Resonator with Varactor
- LC Resonator --> Output Tap --> Amplifier + Filter --> ADC (MCP3202)
- ADC --> SPI --> ESP32 Controller

---

2. Resonator Circuit Design

- The resonator forms the core physical system.
- It must exhibit a clear resonance peak and controllable frequency drift.

LC Resonator Structure

- The resonator is constructed using an inductor and a voltage-controlled capacitor, forming a parallel resonant tank circuit.
- **Basic topology:** Pulse Input is connected to the Inductor (L) and Varactor Diode (C(V)) in parallel, which then connect to GND.

Recommended Component Values

- These values are selected to produce resonance in the 300 kHz – 800 kHz range, which is easily handled by the ESP32 system.
- **Inductor:** Recommended value is L = 10 μH. Example components include shielded RF inductors with a tolerance of $\le$ 5%.
- **Varactor Diode:** Recommended parts are BB909, MV209, or SMV1231. Typical capacitance range is 20 pF – 200 pF.
- **Expected Resonance Frequency:** The resonance frequency is calculated using:

  $$f_0 = \frac{1}{2\pi\sqrt{LC}}$$

- **Example calculation:** If L = 10 μH and C = 100 pF, the result is $f_0 \approx 503$ kHz, which falls within the desired operating range.

---

3. Varactor Bias Circuit

- The varactor diode requires a stable reverse bias voltage, which determines its capacitance.
- **Bias Network:** A 3.3V source connects to a 10 kΩ resistor, which connects to the varactor diode and then to GND. The control voltage can be injected through a bias modulation path.
- **Drift Injection**: Frequency drift is simulated by modulating the bias voltage.
- **PWM-Based Drift:** ESP32 PWM --> RC filter --> bias node. Example filter values are R = 10 kΩ and C = 1 μF. This converts PWM into a slowly varying analog voltage.

---

4. Pulse Generation Circuit

- The pulse generator drives the resonator. The ESP32 communicates with the DAC via SPI.
- **DAC Connection:** Recommended DAC is MCP4921. SPI connections: ESP32 MOSI --> DAC SDI; ESP32 SCK --> DAC SCK; ESP32 CS --> DAC CS. Power connections: VDD = 3.3V; VSS = GND.
- **DAC Output Buffer:** The DAC output must be buffered before driving the resonator to isolate the DAC from the resonator load. Recommended op-amps include TL072, OPA2134, or LM358 (acceptable but lower performance).
- **Buffer Configuration:** Configured as a non-inverting amplifier with a gain of 1.

---

5. Resonator Input Coupling

- The pulse signal must be coupled into the resonator without disturbing the bias network.
- Use AC coupling. The circuit flows from the Pulse Output --> Coupling Capacitor --> Resonator Node. Recommended capacitor value is 100 nF.

---

6. Output Measurement Tap

- The resonator output is measured at the resonator node.
- Measurement must not significantly load the circuit, requiring the use of a high impedance buffer.
  Measurement Buffer flow: Resonator Node --> Op-Amp Buffer --> Measurement Output.

---

7. Signal Conditioning Filter

- The measurement signal is filtered before entering the ADC.
- **Low-Pass Filter:** An example RC filter uses R = 10 kΩ and C = 10 nF to reduce high-frequency noise.

---

8. ADC Interface

- Recommended ADC is the MCP3202.
- ADC SPI Connections: ESP32 MOSI --> ADC DIN; ESP32 MISO --> ADC DOUT; ESP32 SCK --> ADC CLK; ESP32 CS --> ADC CS.
- ADC Input Range: The typical input range is 0 – 3.3V. Signal conditioning must ensure the resonator output stays within this range.

---

9. Power Supply Design & Grounding Strategy

- A stable power supply is essential. Recommended voltage regulators include the AMS1117-3.3.
- **Decoupling Capacitors:** Each IC should have a 100 nF ceramic capacitor and a 10 μF electrolytic capacitor to stabilize the supply voltage.
- **Grounding Strategy:** Separate analog and digital grounds where possible, using a single connection point to reduce digital noise interference.

---

10. Optional Amplification Stage

- If the resonator output amplitude is small, an amplification stage may be required.
- **Example configuration:** Non-inverting amplifier with a gain of 2 – 5, utilizing an op-amp such as the OPA2134.

---

11. Hardware Validation Tests & Performance Targets

Before integrating AI control, the hardware must pass these tests:

- **Test 1** — Resonance Sweep: Perform a frequency sweep to expect a clear resonance peak and measurable amplitude variation.
- **Test 2** — Drift Injection: Apply varying bias voltage to expect a measurable frequency shift.
- **Test 3** — Pulse Injection: Inject pulses and observe resonator ringing to expect a visible oscillatory response.

Hardware Performance Targets:

- **Resonance frequency:** 300 kHz – 800 kHz
- **Drift range:** ±5%
- **Measurement noise:** <5%
- **Control loop latency:** <50 ms

Software, Firmware, and Experimental Architecture

After hardware validation, the next stage is implementing the firmware architecture for pulse generation and measurement acquisition, connecting the circuit to the reinforcement learning control system.

1. Embedded Firmware Architecture

The firmware running on the ESP32 acts as the real-time hardware controller, defining how the controller actually operates the hardware. It is responsible for generating waveform pulses, sampling resonator response, communicating with the Python control system, and maintaining deterministic timing.

- **Waveform Generation Module**: Generates waveform samples, transmits them to the DAC, and controls pulse timing. Functions include generate_waveform(), load_waveform_buffer(), start_pulse(), and stop_pulse(). Waveforms are stored as amplitude sample arrays (e.g., Gaussian: [0, 5, 12, 30, 60, 100, 140, 160, 140, 100, 60, 30, 12, 5, 0]) and streamed via SPI.
- **Measurement Module:** Acquires signals by triggering ADC sampling, collecting multiple samples, averaging them, and returning an amplitude estimate. Example functions are start_adc_sampling(), read_adc_samples(), and compute_average(). Oversampling 64 ADC samples improves measurement stability.

- **Timing Control Module:** Must use hardware timers, not software delays, to schedule pulses, synchronize DAC updates, and ensure consistent timing. Uses ESP32 hardware timers, SPI controller, and optional DMA buffers.
- **Communication Module:** Handles UART Serial communication with Python at a typical baud rate of 921600 baud. Functions include parse_command(), send_measurement(), and acknowledge_command().

---

2. Communication Protocol Specification

Without a protocol, the hardware–AI interface will become messy. The protocol defines a clear text-based structure between the firmware and control system.

- **Command Structure:** COMMAND parameter1 parameter2 parameter3.
- **Set Pulse Parameters:** SET_PULSE frequency amplitude duration waveform (e.g., SET_PULSE 500000 0.8 20 gaussian).
- **Execute Pulse:** EXECUTE_PULSE triggers generation.
- **Request Measurement:** GET_MEASUREMENT yields a response formatted as MEASURE amplitude_value (e.g., MEASURE 0.92).
- **Frequency Sweep:** START_SWEEP start_freq end_freq step returns a sequence of amplitude measurements.

---

3. Python Control Software Architecture

The Python system acts as the control layer between hardware and AI, orchestrating experiments and interactions.

- **Hardware Interface Module:** Communicates with the ESP32 using pyserial via functions like send_command(), read_response(), set_pulse(), and get_measurement().
- **Experiment Controller:** Controls flow by performing frequency sweeps, collecting measurements, and running optimization loops via functions such as run_frequency_sweep(), run_rl_optimization(), and collect_dataset().
- **Data Logging System:** Records all experiment data, including timestamp, pulse parameters, and reward values, into CSV files, pandas DataFrames, or SQLite databases.

---

4. Reinforcement Learning Implementation

The RL system learns optimal pulse parameters to maximize resonator response.

- **State Representation:** Variables may include previous pulse frequency and amplitude, recent measurements, and estimated resonance frequency.
- **Action Space:** Actions modify pulse parameters like frequency, amplitude, duration, and waveform type.
- **Reward Function:** Basic reward = measured_amplitude, with optional refinements including power penalties.
- **Algorithm Selection:** PPO (Proximal Policy Optimization) is recommended for initial implementation due to stability, alongside options like DDPG or SAC.
- **Training Loop:** Continuously loops through: 1. observe state --> 2. choose action --> 3. apply pulse --> 4. measure response --> 5. compute reward --> 6. update policy.

---

5. Simulation Environment Implementation

A detailed software simulation is required before connecting real hardware.

- **Resonator Model:** Response modeled using the Lorentzian function:

  $$A(f) = \frac{1}{1 + Q^2 \left(\frac{f-f_0}{f_0}\right)^2}$$

- **Drift Model:** Resonant frequency varies over time (e.g., $f_0 = f_0 + random_noise$) via random walk, sinusoidal variation, or filtered noise.
- **Measurement Noise:** Adds Gaussian noise (2–5% signal amplitude) to true values.

---

6. GUI Dashboard Design

Provides real-time visualization of system behavior.

- **Required Dashboard Panels:** Resonator Frequency Response (amplitude vs frequency), Pulse Parameter Evolution, Reward Convergence Plot, Drift Tracking, and Control Loop Latency.
- **Implementation Frameworks:** Plotly, Dash, PyQt, or matplotlib.

---

7. Experimental Methodology & Data Analysis Metrics

Necessary for scientific reproducibility and quantitative evaluation.

- Experiment 1 (Resonance Identification): Perform a sweep to find a clear resonance peak.
- Experiment 2 (Drift Tracking): Inject slow drift; expect AI to track frequency changes.
- Experiment 3 (Pulse Optimization): Start with random parameters; expect convergence to the optimal pulse.
- Experiment 4 (Noise Robustness): Increase measurement noise; expect stable optimization.
- Evaluation Metrics: Resonance Estimation Error, Energy Transfer Stability (standard deviation of measured amplitude), Optimization Convergence Speed, and Control Loop Latency.

---

8. Project Directory Structure

To maintain organization, the project repository should follow a clear structure. Example layout:

- firmware/ (esp32_control, waveform_generator, adc_interface)
- python_control/ (hardware_interface, experiment_controller, data_logging)
- rl_training/ (rl_environment, training_scripts, trained_models)
- simulation/ (resonator_model, drift_generator)
- visualization/ (dashboard, plotting_tools)
- data_logs/

---

9. Final System Outcome

If implemented correctly, the system will demonstrate autonomous identification of resonance frequency, real-time adaptation to drift, reinforcement learning-based pulse optimization, and stable energy transfer despite environmental changes. This fully validates the autonomous adaptive control architecture.